


[_ \(http://hl7.org/fhir/R4/searchform.html\)](http://hl7.org/fhir/R4/searchform.html)

[Exchange \(exchange-module.html\)](http://hl7.org/fhir/exchange-module.html) > [RESTful API \(http.html\)](http://hl7.org/fhir/http.html) > **Search**

This page is part of the FHIR Specification (v4.0.1: R4 - Mixed [Normative \(https://confluence.hl7.org/display/HL7/HL7+Balloting\)](https://confluence.hl7.org/display/HL7/HL7+Balloting) and [STU \(https://confluence.hl7.org/display/HL7/HL7+Balloting\)](https://confluence.hl7.org/display/HL7/HL7+Balloting)) in it's permanent home (it will always be available at this URL). The current version which supercedes this version is [5.0.0 \(http://hl7.org/fhir/index.html\)](http://hl7.org/fhir/index.html). For a full list of available versions, see the [Directory of published versions \(http://hl7.org/fhir/directory.html\)](http://hl7.org/fhir/directory.html). Page versions: [R5 \(http://hl7.org/fhir/R5/search.html\)](http://hl7.org/fhir/R5/search.html), [R4B \(http://hl7.org/fhir/R4B/search.html\)](http://hl7.org/fhir/R4B/search.html), **[R4 R3 \(http://hl7.org/fhir/STU3/search.html\)](http://hl7.org/fhir/STU3/search.html)**, [R2 \(http://hl7.org/fhir/DSTU2/search.html\)](http://hl7.org/fhir/DSTU2/search.html)

3.1.1_Search

FHIR Infrastructure (http://www.hl7.org/Special/committees/fiwi/index.cfm) Work Group	Maturity Level (versions.html#maturity): Normative	Standards Status (versions.html#std-process): Normative (versions.html#std-process)
--	---	---

 This page has been approved as part of an [ANSI \(https://www.ansi.org/\)](https://www.ansi.org/) standard. See the [Infrastructure \(ansi-infrastructure.html\)](#) Package for further details.

Searching for resources is fundamental to the mechanics of FHIR. Search operations traverse through an existing set of resources filtering by parameters supplied to the search operation. The text below describes the FHIR search framework, starting with simple cases moving to the more complex. Implementers need only implement the amount of complexity that they require for their implementations.

3.1.1.1 Summary Table

Search Parameter Types	Parameters for all resources	Search result parameters
Number	_id	_sort
Date/DateTime	_lastUpdated	_count
String	_tag	_include
Token	_profile	_revinclude
Reference	_security	_summary
Composite	_text	_total
Quantity	_content	_elements
URI	_list	_contained
Special	_has	_containedType
	_type	

In addition, there is a special search parameters [_query](#) and [_filter](#) that allow for an alternative method of searching, and the parameters [_format](#) and [_pretty](#) [defined for all interactions \(http.html#parameters\)](#).

Also, there is a single page that lists [all the search parameters \(searchparameter-registry.html\)](#). Note that search parameter names are case sensitive, though this specification never defines different parameters with names that differ only in case. Clients SHOULD use correct case, and servers SHALL not define additional parameters with different meanings with names that only differ in case.

There are [several safety issues \(safety.html#search\)](#) associated with the implementation of searching that implementers should always keep in mind.

3.1.1.2 Introduction

In the simplest case, a search is executed by performing a `GET` operation in the RESTful framework:

```
GET [base]/[type]?name=value&...{&_format=[mime-type]}
```

For this RESTful search (see [definition in RESTful API \(http.html#search\)](#)), the parameters are a series of `name=[value]` pairs encoded in the URL or as an `application/x-www-form-urlencoded` submission for a POST:

```
POST [base]/[type]/_search{?[parameters]}{&_format=[mime-type]}
```

The server determines which of the set of resources it serves meet the specific criteria, and returns the results in the HTTP response as a [bundle \(bundle.html\)](#), which includes the resources that are the results of the search. Note that the `_format` parameter works for search [like for other interactions \(http.html#mime-type\)](#).

[Search operations \(http.html#search\)](#) are executed in one of three defined contexts that control which set of resources are being searched:

- A specified resource type: `GET [base]/[type]?parameter(s)`
- A specified [compartment \(compartmentdefinition.html\)](#), perhaps with a specified resource type in that compartment:
`GET [base]/Patient/[id]/[type]?parameter(s)`
- All resource types: `GET [base]?parameter(s)` (parameters common to all types). If the `_type` parameter is included, all other search parameters SHALL be common to all provided types. If `_type` is not included, all parameters SHALL be common to all resource types.

Search operations can also be implemented in [the messaging framework \(messaging.html#search\)](#).

The server determines which of their resources meet the criteria contained in the search parameters as described below. However, the server has the prerogative to return additional search results if it believes them to be relevant. Note: There is a special search for the most relevant context in which the search set is indeterminate: [Patient MPI Search \(patient.html#match\)](#).

Search using `GET` may include sensitive information in the search parameters. Therefore, secure communications and endpoint management are recommended, see [Security Communications \(security.html#http\)](#).

The response to any search operation is always a list of resources in a Bundle. An alternative approach is to [use GraphQL \(graphql.html\)](#).

3.1.1.3 Handling Errors

If a server is unable to execute a search request, it may return an error. A HTTP status code of `403` signifies that the server refused to perform the search, while other `4xx` and `5xx` codes signify that some sort of error has occurred. When the search fails, a server SHOULD return an [OperationOutcome \(operationoutcome.html\)](#) detailing the cause of the failure. Note: An empty search result is not a failure.

In some cases, parameters may cause an error, or might not be able to match anything. For instance:

- A parameter may refer to a non-existent resource e.g. `GET [base]/Observation?subject=101`, where "101" does not exist
- A parameter may refer to a non-existent resource e.g.
`GET [base]/Observation?patient.identifier=http://example.com/fhir/identifier/mrn|123456`, where there is no patient for MRN 123456
- A parameter may refer to an unknown code e.g. `GET [base]/Observation?code=loinc|1234-1`, where the LOINC code "1234-1" is not known to the server

- A parameter may refer to a time that is out of scope e.g. `GET [base]/Condition?onset=le1995` , where the system only has data going back to 2001
- A parameter may use an illegal or unacceptable modifier e.g. `GET [base]/Condition?onset:text=1995` , where the modifier cannot be processed by the server
- A data time parameter may have incorrect format e.g. `GET [base]/Condition?onset=23%20May%202009`
- A parameter may be unknown or unsupported (see below)

Note: Empty parameters are not an error - they are just ignored by the server.

Where the content of the parameter is syntactically incorrect, servers **SHOULD** return an error. However, where the issue is a logical condition (e.g. unknown subject or code), the server **SHOULD** process the search, including processing the parameter - with the result of returning an empty search set, since the parameter cannot be satisfied.

In such cases, the search process **MAY** include an [OperationOutcome](#) ([operationoutcome.html](#)) in the search set that contains additional hints and warnings about the search process. This is included in the search results as an entry with [search mode](#) ([bundle-definitions.html#Bundle.entry.search.mode](#)) = [outcome](#) ([valueset-search-entry-mode.html](#)). Clients can use this information to improve future searches. If, for example, a client performed the following search:

```
GET [base]/Observation?patient.identifier=http://example.com/fhir/identifier/mrn|123456
```

and there is no patient for MRN 123456, the server would return [a bundle with a warning](#) ([bundle-search-warning.html](#)).

Unknown and unsupported parameters

Servers may receive parameters from the client that they do not recognize, or may receive parameters they recognize but do not support (either in general, or for a specific search). In general, servers **SHOULD** ignore unknown or unsupported parameters for the following reasons:

- Various HTTP stacks and proxies may add parameters that aren't under the control of the client
- The client can determine what parameters the server used by examining the self link in the return (see [below](#))

Clients can specify how the server should behave, by using the `prefer` header

- `Prefer: handling=strict`: Client requests that the server return an error for any unknown or unsupported parameter
- `Prefer: handling=lenient`: Client requests that the server ignore any unknown or unsupported parameter

Servers **SHOULD** honor the client's request, but are not required to do so.

3.1.1.4 Standard Parameters

3.1.1.4.1 Parameters for all resources

The following parameters apply [to all resources](#) ([resource.html#search](#)): `_content` , `_id` , `_lastUpdated` , `_profile` , `_query` , `_security` , `_source` , `_tag` . In addition, the search parameter `_text` and `_filter` , (documented below) also applies to all resources (as do the search result parameters).

The search parameter `_id` refers to the logical id of the resource, and can be used when the search context specifies a resource type:

```
GET [base]/Patient?_id=23
```

This search finds the patient resource with the given id (there can only be one resource for a given id). Functionally, this is equivalent to a [simple read operation](#) ([http.html#read](#)):

```
GET [base]/Patient/23
```

However, the search with parameter `_id` returns a bundle with the requested resource, instead of just the resource itself. Additional parameters can be added which may provide additional functionality on top of this base read equivalence (e.g. `_include`). Note that although the `_id` parameter has a type of token, because servers SHALL use exact match with it, there is no system for the `_id` parameter. Note that matches on `_id` are always case sensitive.

The search parameter `_lastUpdated` can be used to select resources based on the last time they were changed:

```
GET [base]/Observation?_lastUpdated=gt2010-10-01
```

This search finds any observations changed since 1-Oct 2010. When this search parameter is used, applications should consider synchronization approaches ([RESTful history \(http.html#history\)](http://hl7.org/fhir/RESTful-history.html#history)) or the [Subscription resource \(subscription.html\)](http://hl7.org/fhir/Subscription-resource.html).

The search parameters `_tag (resource.html#simple-tags)`, `_profile (resource.html#profile-tags)` and `_security (resource.html#security-labels)` parameters search on the equivalent elements in the `meta` element (`resource.html#meta`). For example,

```
GET [base]/Condition?_tag=http://acme.org/codes|needs-review
```

searches for all Condition resources with the tag:

```
{
  "system" : "http://acme.org/codes",
  "code" : "needs-review"
}
```

In the same manner:

```
GET [base]/DiagnosticReport?_profile=http://hl7.org/fhir/StructureDefinition/lipid
GET [base]/DiagnosticReport?_profile=Profile/lipid
```

restricts the search to only DiagnosticReport resources that are tagged as conforming to a particular profile. The second reference is relative and refers a local profile on the same server.

`_tag (resource.html#simple-tags)`, and `_security (resource.html#security-labels)` parameters are token types (see [below](#)), and `_profile (resource.html#profile-tags)` is a [reference](#) search parameter.

3.1.1.4.2 Parameters for each resource

In addition to the `_id` parameter which exists for all resources, each FHIR resource type defines its own set of search parameters with their names, types, and meanings. These search parameters are on the same page as the resource definitions, and are also published as part of the standard Capability statement ([XML \(capabilitystatement-base.xml.html\)](http://hl7.org/fhir/capabilitystatement-base.xml.html) or [JSON \(capabilitystatement-base.json.html\)](http://hl7.org/fhir/capabilitystatement-base.json.html)).

In general, the defined search parameters correspond to a single element in the resource, but this is not required, and some search parameters refer to the same type of element in multiple places, or refer to derived values.

Some search parameters defined by resources are associated with more than one path in a resource. This means that the search parameter matches if any of the paths contain matching content. If a path matches, the whole resource is returned in the search results. The client may have to examine the resource to determine which path contains the match.

Servers are not required to implement any of the standard search parameters (except for the `_id` parameter described above). Servers may also define

their own parameters.

3.1.1.4.3 Search Parameter Types

Each search parameter is defined by a type that specifies how the search parameter behaves. These are the defined parameter types:

<u>number</u>	Search parameter SHALL be a number (a whole number, or a decimal).
<u>date</u>	Search parameter is on a date/time. The date format is the standard XML format, though other formats may be supported.
<u>string</u>	Search parameter is a simple string, like a name part. Search is case-insensitive and accent-insensitive. May match just the start of a string. String parameters may contain spaces.
<u>token</u>	Search parameter on a coded element or identifier. May be used to search through the text, display, code and code/codesystem (for codes) and label, system and key (for identifier). Its value is either a string or a pair of namespace and value, separated by a " ", depending on the modifier used.
<u>reference</u>	A reference to another resource (Reference or canonical).
<u>composite</u>	A composite search parameter that combines a search on two values together.
<u>quantity</u>	A search parameter that searches on a quantity.
<u>uri</u>	A search parameter that searches on a URI (RFC 3986).
<u>special</u>	Special logic applies to this parameter per the description of the search parameter.

The search parameters can also append "modifiers" that control their behavior. The kinds of modifiers that available is dependent on the type of parameter being modified.

3.1.1.4.4 Modifiers

Parameters are defined per resource. Parameter names may specify a modifier as a suffix. The modifiers are separated from the parameter name by a colon. Modifiers are:

- For all parameters (except combination): `:missing` ; e.g. `gender:missing=true` (or false). Searching for `gender:missing=true` will return all the resources that don't have a value for the gender parameter (which usually equates to not having the relevant element in the resource). Searching for `gender:missing=false` will return all the resources that have a value for the `gender` parameter. For simple data type elements, `:missing=true` will match on all elements where either the underlying element is omitted or where the element is present with extensions but no `@value` is specified
- For string: `:exact` returns results that match the entire supplied parameter, including casing and combining characters. Note that the handling of (extended) [grapheme clusters](http://unicode.org/reports/tr29/#Grapheme_Cluster_Boundaries) (http://unicode.org/reports/tr29/#Grapheme_Cluster_Boundaries) is within the discretion of the server, i.e. the server decides if a string search parameter matches on canonically equivalent characters, or matches on the actual used Unicode code points, or `:contains` (case insensitive and combining character-insensitive, search text matched anywhere in the string), instead of the default behavior (case insensitive and accent-insensitive, partial matches at the start of the string).
- For token: `:text` (the match does a partial searches on the text portion of a CodeableConcept or the display portion of a Coding), instead of the default search which uses codes. Other defined modifiers are `:in` , `:below` , `:above` and `:not-in` which are described below.
- For reference: `: [type]` where [type] is the name of a type of resource, `:identifier`, and, for some parameters, `:above` and `:below`
- For uri: `:below` , `:above` indicate that instead of an exact match, either the search term left-matches the value, or vice-versa.

Server SHALL reject any search request that contains is suffixed by a modifier that the server does **not** support for that parameter. For example, if the server supports the `name` search parameter, but not the `:exact` modifier on the name, it should reject a search with the parameter `name:exact=Bill` , using an HTTP 400 error with an [OperationOutcome \(operationoutcome.html\)](#) with a [clear error message \(operationoutcome-example-searchfail.html\)](#).

3.1.1.4.5 Prefixes

For the ordered parameter types of number, date, and quantity, a prefix to the parameter value may be used to control the nature of the matching. To avoid URL escaping and visual confusion, the following prefixes are used:

eq	the value for the parameter in the resource is equal to the provided value	the range of the search value fully contains the range of the target value
ne	the value for the parameter in the resource is not equal to the provided value	the range of the search value does not fully contain the range of the target value
gt	the value for the parameter in the resource is greater than the provided value	the range above the search value intersects (i.e. overlaps) with the range of the target value
lt	the value for the parameter in the resource is less than the provided value	the range below the search value intersects (i.e. overlaps) with the range of the target value
ge	the value for the parameter in the resource is greater or equal to the provided value	the range above the search value intersects (i.e. overlaps) with the range of the target value, or the range of the search value fully contains the range of the target value
le	the value for the parameter in the resource is less or equal to the provided value	the range below the search value intersects (i.e. overlaps) with the range of the target value or the range of the search value fully contains the range of the target value
sa	the value for the parameter in the resource starts after the provided value	the range of the search value does not overlap with the range of the target value, and the range above the search value contains the range of the target value
eb	the value for the parameter in the resource ends before the provided value	the range of the search value does overlap not with the range of the target value, and the range below the search value contains the range of the target value
ap	the value for the parameter in the resource is approximately the same to the provided value. Note that the recommended value for the approximation is 10% of the stated value (or for a date, 10% of the gap between now and the date), but systems may choose other values where appropriate	the range of the search value overlaps with the range of the target value

If no prefix is present, the prefix **eq** is assumed. Note that the way search parameters operate is not the same as the way the operations on two numbers work in a mathematical sense. **sa** (**starts-after**) and **eb** (**ends-before**) are not used with integer values but are used for decimals.

For each prefix above, two interpretations are provided - the simple intent of the prefix and the interpretation of the parameter when applied to ranges. The range interpretation is provided for decimals and dates. Searches are always performed on values that are implicitly or explicitly a range. For instance, the number 2.0 has an implicit range of 1.95 to 2.05, and the date 2015-08-12 has an implicit range of all the time during that day. If the target value is a Range ([datatypes.html#range](#)), a Period ([datatypes.html#period](#)), or a Timing ([datatypes.html#timing](#)), then the target is explicitly a range. Three ranges are identified:

range of the value	The limits implied by the precision of the value	The number 2.0 has a range of 1.95 to 2.05 The date 2015-08-12 has a range from 00:00 to 00:00 exclusive
range below the value	Up to the specified value	The range below 2.0 includes any value less or equal to <2.00000000000000000000 The range before 2015-08-12T05:23:45 includes any time up to 2015-08-12T05:23:45.0000000000000000
range above the value	The specified value and up	The range above 2.0 includes any value greater or equal to <2.00000000000000000000 The range after 2015-08-12T05:23:45 includes any time after 2015-08-12T05:23:45.0000000000000000

The proper use of these ranges is discussed further below.

3.1.1.4.6 number

Searching on a simple numerical value in a resource. Examples:

[parameter]=100	Values that equal 100, to 3 significant figures precision, so this is actually searching for values in the range [99.5 ... 100.5)
[parameter]=100.00	Values that equal 100, to 5 significant figures precision, so this is actually searching for values in the range [99.995 ... 100.005)
[parameter]=1e2	Values that equal 100, to 1 significant figures precision, so this is actually searching for values in the range [95 ... 105)
[parameter]=lt100	Values that are less than exactly 100
[parameter]=le100	Values that are less or equal to exactly 100
[parameter]=gt100	Values that are greater than exactly 100
[parameter]=ge100	Values that are greater or equal to exactly 100
[parameter]=ne100	Values that are not equal to 100 (actually, in the range 99.5 to 100.5)

Notes about searching on Numbers:

- When a number search is used against a resource element that stores a simple integer (e.g. [ImmunizationRecommendation.recommendation.doseNumber](#) ([immunizationrecommendation-definitions.html#ImmunizationRecommendation.recommendation.doseNumber_x](#))), and the search parameter is not expressed using the exponential forms, and does not include any non-zero digits after a decimal point, the significance issues cancel out and searching is based on exact matches. Note that if there are non-zero digits after a decimal point, there cannot be any matches
- When a comparison prefix in the set `lt`, `gt`, `le`, `sa` & `eb` is provided, the implicit precision of the number is ignored, and they are treated as if they have arbitrarily high precision
- The way search parameters operate in resources is not the same as whether two numbers are equal to each other in a mathematical sense
- Searching on decimals involves an implicit range. The number of significant digits of the implicit range is the number of digits specified in the search parameter value, excluding leading zeros. So 100 and 1.00e2 both have the same number of significant digits - three

Here are some example searches:

Search	Description
GET [base]/RiskAssessment?probability=gt0.8	Search for all the Risk Assessments with probability great than 0.8 (could also be <code>probability=gt8e-1</code> using exponential form)
GET [base]/ImmunizationRecommendation?dose-number=2	Search for any immunization recommendation recommending a second dose

3.1.1.4.7 date

A date parameter searches on a date/time or period. As is usual for date/time related functionality, while the concepts are relatively straight-forward, there are a number of subtleties involved in ensuring consistent behavior.

The date parameter format is `yyyy-mm-ddThh:mm:ss[Z|(+|-)hh:mm]` (the standard XML format).

Technically, this is any of the [date](#) ([datatypes.html#date](#)), [dateTime](#) ([datatypes.html#dateTime](#)), and [instant](#) ([datatypes.html#instant](#)) data types; e.g. Any degree of precision can be provided, but it SHALL be populated from the left (e.g. can't specify a month without a year), except that the minutes SHALL be present if an hour is present, and you SHOULD provide a time zone if the time part is present. Note: Time can consist of hours and minutes with no seconds, unlike the XML Schema `dateTime` type. Some user agents may escape the `:` characters in the URL, and servers SHALL handle this correctly.

Date parameters may be used with the following data types:

<u>date (datatypes.html#date)</u>	The range of the value is the day, month, or year as specified
<u>dateTime (datatypes.html#dateTime)</u>	The range of the value as defined above; e.g. For example, the date 2013-01-10 specifies all the time from 00:00 on 10-Jan 2013 to immediately before 00:00 on 11-Jan 2013
<u>instant (datatypes.html#instant)</u>	An instant is considered a fixed point in time with an interval smaller than the precision of the system, i.e. an interval with an effective width of 0
<u>Period (datatypes.html#Period)</u>	Explicit, though the upper or lower bound might not actually be specified in resources.
<u>Timing (datatypes.html#Timing)</u>	the specified scheduling details are ignored and only the outer limits matter. For instance, a schedule that specifies every second day between 31-Jan 2013 and 24-Mar 2013 includes 1-Feb 2013, even though that is on an odd day that is not specified by the period. This is to keep the server load processing queries reasonable.

Implicitly, a missing lower boundary is "less than" any actual date. A missing upper boundary is "greater than" any actual date. The use of the prefixes:

[parameter]=eq2013-01-14	✓ 2013-01-14T00:00 matches (obviously) ✓ 2013-01-14T10:00 matches ✗ 2013-01-15T00:00 does not match - it's not in the range
[parameter]=ne2013-01-14	✓ 2013-01-15T00:00 matches - it's not in the range ✗ 2013-01-14T00:00 does not match - it's in the range ✗ 2013-01-14T10:00 does not match - it's in the range
[parameter]=lt2013-01-14T10:00	✓ 2013-01-14 matches, because it includes the part of 14-Jan 2013 before 10am
[parameter]=gt2013-01-14T10:00	✓ 2013-01-14 matches, because it includes the part of 14-Jan 2013 after 10am
[parameter]=ge2013-03-14	✓ "from 21-Jan 2013 onwards" is included because that period may include times after 14-Mar 2013
[parameter]=le2013-03-14	✓ "from 21-Jan 2013 onwards" is included because that period may include times before 14-Mar 2013
[parameter]=sa2013-03-14	✓ "from 15-Mar 2013 onwards" is included because that period starts after 14-Mar 2013 ✗ "from 21-Jan 2013 onwards" is not included because that period starts before 14-Mar 2013 ✗ "before and including 21-Jan 2013" is not included because that period starts (and ends) before 14-Mar 2013
[parameter]=eb2013-03-14	✗ "from 15-Mar 2013 onwards" is not included because that period starts after 14-Mar 2013 ✗ "from 21-Jan 2013 onwards" is not included because that period starts before 14-Mar 2013, but does not end before it ✓ "before and including 21-Jan 2013" is included because that period ends before 14-Mar 2013

[parameter]=ap2013-03-14	14-Mar 2013 is included - as it exactly matches ✖ 21-Jan 2013 is not included because that is near 14-Mar 2013 ✖ 15-Jun 2015 is not included - as it is not near 14-Mar 2013. Note that the exact value here is at the discretion of the system
--------------------------	---

Other notes:

- When the date parameter is not fully specified, matches against it are based on the behavior of intervals, where:
 - Dates with only the year specified are equivalent to an interval that starts at the first instant of January 1st to the last instant of December 31st, e.g. 2000 is equivalent to an interval of [2000-01-01T00:00, 2000-12-31T23:59].
 - Dates with the year and month are equivalent to an interval that starts at the first instant of the first day of the month and ends on the last instant of the last day of the month, e.g. 2000-04 is equivalent to an interval of [2000-04-01T00:00, 2000-04-30T23:59].
- Where possible, the system should correct for time zones when performing queries. Dates do not have time zones, and time zones should not be considered. Where both search parameters and resource element date times do not have time zones, the servers local time zone should be assumed.

To search for all the procedures in a patient compartment that occurred over a 2-year period:

```
GET [base]/Patient/23/Procedure?date=ge2010-01-01&date=le2011-12-31
```



Managing time zones and offsets and their impact on search is a very difficult area. The FHIR implementation community is still investigating and debating the best way to handle time zones. Implementation guides may make additional rules in this regard.

Future versions of this specification may impose rules around the use of time zones with dates. Implementers and authors of implementation guides should be aware of ongoing work in this area.

Implementer feedback is welcome [on the issue tracker](http://hl7.org/fhir-issues) (http://hl7.org/fhir-issues) or [chat.fhir.org](https://chat.fhir.org/#narrow/stream/4-implementers) (https://chat.fhir.org/#narrow/stream/4-implementers).

3.1.1.4.8 string

For a simple string search, a string parameter serves as the input for a search against sequences of characters. This search is insensitive to casing and included combining characters, like accents or other diacritical marks. Punctuation and non-significant whitespace (e.g. repeated space characters, tab vs space) should also be ignored. By default, a field matches a string query if the value of the field equals or starts with the supplied parameter value, after both have been normalized by case and combining characters. Therefore, the default string search only operates on the base characters of the string parameter. The :contains modifier returns results that include the supplied parameter value anywhere within the field being searched. The :exact modifier returns results that match the entire supplied parameter, including casing and accents.

Examples:

[base]/Patient?given=eve	Any patients with a name containing a given part with "eve" at the start of the name. This would include patients with the given name "Eve", "Evelyn".
[base]/Patient?given:contains=eve	Any patients with a name with a given part containing "eve" at any position. This would include patients with the given name "Eve", "Evelyn", and also "Severine".
[base]/Patient?given:exact=Eve	Any patients with a name with a given part that is exactly "Eve". Note: This would not include patients with the given name "eve" or "EVE".

An additional modifier `:text` can be used to specify a search with advanced text handling (see [below](#)) though only a few servers are expected to offer this facility.

When a string search parameter refers to the types [HumanName \(datatypes.html#HumanName\)](#) and [Address \(datatypes.html#Address\)](#), the search covers the elements of type string, and does not cover elements such as `use` and `period`. For robust search, servers should search the parts of a family name independently. E.g. searching either "Carreno" or "Quinones" should match a family name of "Carreno Quinones". HL7 affiliates may make more specific recommendations about how search should work in their specific culture.

It is at the discretion of the server whether to pre-process names, addresses, and contact details to remove separator characters prior to matching in order to ensure more consistent behavior. For example, a server might remove all spaces and `-` characters from phone numbers. What is most appropriate varies depending on culture and context. A server may also use a free-text style searches on this property to achieve the best results.

When searching whole names and addresses (not parts), servers may also use flexible match or a free-text style searches on names to achieve the best results.

3.1.1.4.9 uri

The `uri` parameter refers to an element that contains a URI ([RFC 3986](#) <https://tools.ietf.org/html/rfc3986>). By default, matches are precise (e.g. case, accent, and escape) sensitive, and the entire URI must match. The modifier `:above` or `:below` can be used to indicate that partial matching is used. For example:

```
GET [base]/ValueSet?url=http://acme.org/fhir/ValueSet/123
GET [base]/ValueSet?url:below=http://acme.org/fhir/
GET [base]/ValueSet?url:above=http://acme.org/fhir/ValueSet/123/_history/5
GET [base]/ValueSet?url=urn:oid:1.2.3.4.5
```

- The first line is a request to find any value set with the exact url "http://acme.org/fhir/ValueSet/123"
- The second line performs a search that will return any value sets that have a URL that starts with "http://acme.org/fhir/"
- The third line shows the converse - search for any value set above a given specific URL. This will match on any value set with the specified URL, but also on http://acme.org/ValueSet/123. Note that there are not many use cases where `:above` is useful as compared to the `:below` search
- The fourth line shows an example of searching by an OID. Note that the `:above` and `:below` modifiers only apply to URLs, and not URNs such as OIDs

The search type `uri` is used with elements of type [uri \(datatypes.html#uri\)](#) and [url \(datatypes.html#url\)](#). The type [reference](#) is used for the types [Reference \(references.html#Reference\)](#) and [canonical \(datatypes.html#canonical\)](#). Note that for `uri` parameters that refer to the [Canonical URLs \(references.html#canonical\)](#) of the conformance and knowledge resources (e.g. [StructureDefinition \(structuredefinition.html\)](#), [ValueSet \(valueset.html\)](#), [PlanDefinition \(plandefinition.html\)](#) etc), servers SHOULD support searching by canonical references, and SHOULD support automatically detecting a `[version]` portion as part of the search parameter, and interpreting that portion as a search on the version.

3.1.1.4.10 token

A token type is a parameter that provides a close to exact match search on a string of characters, potentially scoped by a URI. It is mostly used against a code or identifier data type where the value may have a URI that scopes its meaning, where the search is performed against the pair from a Coding or an Identifier. Tokens are also used against other fields where exact matches are required - uris, booleans, and [ContactPoints \(datatypes.html#ContactPoint\)](#). In these cases, the URI portion is not used.

For tokens, matches are literal (e.g. not based on [subsumption \(codesystem.html#subsumption\)](#) or other code system features). Match is case sensitive unless the underlying semantics for the context indicate that the token should be interpreted case-insensitively (see, e.g. [CodeSystem.caseSensitive \(codesystem-definitions.html#CodeSystem.caseSensitive\)](#)). Note that matches on `_id` are always case sensitive. If the underlying data type is [string \(datatypes.html#string\)](#), then the search is not case sensitive.

Note: There are many challenging issues around case sensitivity and token searches. Some code systems are case sensitive (e.g. UCUM) while others are known not to be. For many code systems, it's ambiguous. Other kinds of values are also ambiguous. When in doubt, servers **SHOULD** treat tokens in a case-insensitive manner, on the grounds that including undesired data has less safety implications than excluding desired behavior. Clients **SHOULD** always use the correct case when possible, and allow for the server to perform case-insensitive matching.

To use subsumption-based logic, use the modifiers below, or list all the codes in the hierarchy. The syntax for the value is one of the following:

- **[parameter]=[code]** : the value of **[code]** matches a Coding.code or Identifier.value irrespective of the value of the system property
- **[parameter]=[system]|[code]** : the value of **[code]** matches a Coding.code or Identifier.value, and the value of **[system]** matches the system property of the Identifier or Coding
- **[parameter]=|[code]** : the value of **[code]** matches a Coding.code or Identifier.value, and the Coding/Identifier has no system property
- **[parameter]=[system]|** : any element where the value of **[system]** matches the system property of the Identifier or Coding

Notes:

- The namespace URI and code both must be escaped correctly. If a system is not applicable (e.g. an element of type uri ([datatypes.html#uri](#)), then just the form **[parameter]=[code]** is used.
- For token parameters on elements of type ContactPoint ([datatypes.html#ContactPoint](#)), uri ([datatypes.html#uri](#)), or boolean ([datatypes.html#boolean](#)), the presence of the pipe symbol SHALL NOT be used - only the **[parameter]=[code]** form is allowed
-

Token search parameters are used for the following data types:

Data Type	URI	Code	Comments
<u>Coding</u> (datatypes.html#Coding)	Coding.system	Coding.code	
<u>CodeableConcept</u> (datatypes.html#CodeableConcept)	CodeableConcept.coding.system	CodeableConcept.coding.code	Matches against any coding in the CodeableConcept
<u>Identifier</u> (datatypes.html#Identifier)	Identifier.system	Identifier.value	Clients can search by type not system using the :of-type modifier, see below. To search on a CDA II.root - which may appear in either Identifier.system or Identifier.value , use the syntax identifier= [root],[root]
<u>ContactPoint</u> (datatypes.html#ContactPoint)		ContactPoint.value	At the discretion of the server, token searches on ContactPoint may use special handling, such as ignoring punctuation, performing partial searches etc.
<u>code</u> (datatypes.html#code)	(implicit)	code	the system is defined in the value set (though it's not usually needed)
<u>boolean</u> (datatypes.html#boolean)		boolean	The implicit system for boolean values is http://hl7.org/fhir/special-values (valueset-special-values.html), but this is never actually used
<u>uri</u> (datatypes.html#uri)		uri	
<u>string</u> (datatypes.html#string)	n/a	string	Token is sometimes used for string to indicate that exact matching is the correct default search strategy

Note: The use of token search parameters for boolean fields: the boolean values "true" and "false" are also represented as formal codes in the Special Values ([valueset-special-values.html](#)) code system, which is useful when boolean values need to be represented in a Coding ([datatypes.html#coding](#)) data

type. The namespace for these codes is <http://hl7.org/fhir/special-values>, though there is usually no reason to use this, as a simple true or false is sufficient.

Modifiers:

Modifier	Use
:text	The search parameter is processed as a string that searches text associated with the code/value - either <i>CodeableConcept.text</i> , <i>Coding.display</i> , or <i>Identifier.type.text</i> . In this case, the search functions as a normal string search
:not	Reverse the code matching described in the paragraph above: return all resources that do not have a matching item. Note that this includes resources that have no value for the parameter - e.g. ?gender:not=male includes all patients that do not have gender = male, including patients that do not have a gender at all
:above	The search parameter is a concept with the form <code>[system] [code]</code> , and the search parameter tests whether the coding in a resource subsumes (codesystem.html#subsumption) the specified search code. For example, the search concept has an is-a relationship with the coding in the resource, and this includes the coding itself.
:below	the search parameter is a concept with the form <code>[system] [code]</code> , and the search parameter tests whether the coding in a resource is subsumed by the specified search code. For example, the coding in the resource has an is-a relationship with the search concept, and this includes the coding itself.
:in	The search parameter is a URI (relative or absolute) that identifies a value set, and the search parameter tests whether the coding is in the specified value set (valueset.html). The reference may be literal (to an address where the value set can be found) or logical (a reference to <code>ValueSet.url</code>). If the server can treat the reference as a literal URL, it does, else it tries to match known logical <code>ValueSet.url</code> values.
:not-in	The search parameter is a URI (relative or absolute) that identifies a value set, and the search parameter tests whether the coding is not in the specified value set.
:of-type	The search parameter has the format <code>system code value</code> , where the system and code refer to a <code>Identifier.type.coding.system</code> and <code>.code</code> , and match if any of the type codes match. All 3 parts must be present

Most servers will only process value sets that are already known/registered/supported internally. However, servers can elect to accept any valid reference to a value set. Servers may elect to consider concept mappings when testing for subsumption relationships.

Example searches:

Search	Description
GET [base]/Patient?identifier=http://acme.org/patient 2345	Search for all the patients with an identifier with key = "2345" in the system "http://acme.org/patient"
GET [base]/Patient?gender=male	Search for any patient with a gender that has the code "male"
GET [base]/Patient?gender:not=male	Search for any patient with a gender that does not have the code "male". Note that for <code>:not</code> , the search does not return any resources that have a gen
GET [base]/Composition?section=48765-2	Search for any Composition that contains an Allergies and adverse reaction section
GET [base]/Composition?section:not=48765-2	Search for any Composition that does not contain an Allergies and adverse reaction section. Note that this search does not return "any document that has a section that is not an Allergies and adverse reaction section" (e.g. in the presence of multiple possible matches, the <i>negation</i> applies to the set, not each individual entry)

```
GET [base]/Patient?active=true
```

Search for any patients that are active

```
GET [base]/Condition?code=http://acme.org/conditions/codes|ha125
```

Search for any condition with a code "ha125" in the code system "http://acme.org/conditions/codes"

```
GET [base]/Condition?code=ha125
```

Search for any condition with a code "ha125". Note that there is not often any useful overlap in literal symbols between code systems, so the previous example is generally preferred

```
GET [base]/Condition?code:text=headache
```

Search for any Condition with a code that has a text "headache" associated with it (either in the text, or a display)

```
GET [base]/Condition?code:in=http%3A%2F%2Fsnomed.info%2Fsc%3Ffhir_vs%3Disa%2F126851005
```

Search for any condition in the SNOMED CT value set "http://snomed.info/sct?fhir_vs=isa/126851005" that includes all descendants of "Neoplasm of liver"

```
GET [base]/Condition?code:below=126851005
```

Search for any condition that is subsumed by the SNOMED CT Code "Neoplasm of liver". Note: This is the same outcome as the previous search

```
GET [base]/Condition?code:in=http://acme.org/fhir/ValueSet/cardiac-conditions
```

Search for any condition that is in the institutions list of cardiac conditions

```
GET [base]/Patient?identifier:otype=http://terminology.hl7.org/CodeSystem/v2-0203|MR|446053
```

Search for the Medical Record Number 446053 - this is useful where the system id for the MRN is not known

3.1.1.4.10.1 Searching Mime Types

The `:below` modifier is also very useful with searching mime types, such as for [DocumentReference.contentType \(documentreference.html\)](#) which refers to [Attachment.contentType \(datatypes.html#Attachment\)](#). A simple search such as:

```
GET [base]/DocumentReference?contentType=text/xml
```

will miss documents with a mime type such as `text/xml; charset=UTF-8`. This search will find all text/xml documents:

```
GET [base]/DocumentReference?contentType:below=text/xml
```

For ease of processing on the server, servers are only required to support `:below` on the base part of the mime type; servers are not required to sort between different parameters and do formal subsumption logic.

3.1.1.4.11 quantity

A quantity parameter searches on the [Quantity \(datatypes.html#Quantity\)](#) data type. The syntax for the value follows the form:

- **[parameter]=[prefix][number][system][code]** matches a quantity with the given unit

The prefix is optional, and is as described [above](#), both regarding how precision and comparator/range operators are interpreted. Like a number parameter, the number part of the search value can be a decimal in exponential format. The `system` and `code` follow the same pattern as [token parameters](#) are also optional. Example searches:

Search	Description
GET [base]/Observation?value-quantity=5.4 http://unitsofmeasure.org mg	Search for all the observations with a value of 5.4(+/-0.05) mg where mg is understood as a UCUM unit (<code>system</code> / <code>code</code>)
GET [base]/Observation?value-quantity=5.40e-3 http://unitsofmeasure.org g	Search for all the observations with a value of 0.0054(+/-0.000005) g where g is understood as a UCUM unit (<code>system</code> / <code>code</code>)
GET [base]/Observation?value-quantity=5.4 mg	Search for all the observations with a value of 5.4(+/-0.05) mg where the unit - either the code (<code>code</code>) or the stated human unit (<code>unit</code>) are "mg"
GET [base]/Observation?value-quantity=5.4	Search for all the observations with a value of 5.4(+/-0.05) irrespective of the unit
GET [base]/Observation?value-quantity=le5.4 http://unitsofmeasure.org mg	Search for all the observations where the value of is less than 5.4 mg exactly where mg is understood as a UCUM unit
GET [base]/Observation?value-quantity=ap5.4 http://unitsofmeasure.org mg	Search for all the observations where the value of is about 5.4 mg where mg is understood as a UCUM unit (typically, within 10% of the value - see above)

Specifying a system and a code for the search implies that the search is based on a particular code system - usually [UCUM](http://unitsofmeasure.org) (<http://unitsofmeasure.org>), and that a precise (and potentially canonical) match is desired. In this case, it is inappropriate to search on the human display for the unit, which can be is uncontrolled and may unpredictable.

The search processor may choose to perform a search based on [canonical units \(datatypes.html#quantity\)](#) (e.g. any value where the units can be converted to a value in mg in the case above). For example, an observation may have a value of 23 mm/hr . This is equal to 0.23 m/hr . The search processor can choose to normalise all the values to a canonical unit such as 6.4e-6 m/sec , and convert search terms to the same units (m/sec). Such conversions can be performed based on the semantics defined in [UCUM](http://unitsofmeasure.org) (<http://unitsofmeasure.org>).

3.1.1.4.12 reference

A reference parameter refers to [references between resources \(references.html\)](#). For example, find all Conditions where the subject reference is a particular patient, where the patient is selected by name or identifier. The interpretation of a *reference* parameter is either:

- **[parameter]=[id]** the logical [id] of a resource using a local reference (i.e. a relative reference)
- **[parameter]=[type]/[id]** the logical [id] of a resource of a specified type using a local reference (i.e. a relative reference), for when the reference can point to different types of resources (e.g. [Observation.subject \(observation-definitions.html#Observation.subject\)](#))
- **[parameter]=[url]** where the [url] is an absolute URL - a reference to a resource by its absolute location, or by it's canonical URL

Note: A relative reference resolving to the same value as a specified absolute URL, or vice versa, qualifies as a match. For example, if the search parameter value is Patient/123, then this will find references like this:

```
<patient>
  <reference value="Patient/123"/>
</patient>
```

If the server base address is `http://example.org/fhir`, then the full URL for that reference is `http://example.org/fhir/Patient/123`, which means that the search term also matches patient references like this:

```
<patient>
  <reference value="http://example.org/fhir/Patient/123"/>
</patient>
```

In addition, searching for `reference=http://example.org/fhir/Patient/123` will also match both references.

Some references may point to more than one type of resource; e.g. `subject: Reference(Patient|Group|Device|..)`. In these cases, multiple resources may have the same logical identifier. Servers SHOULD reject a search where the logical id refers to more than one matching resource across different types. In order to allow the client to perform a search in these situations the type is specified explicitly:

```
GET [base]/Observation?subject=Patient/23
```

This searches for any observations where the subject refers to the patient resource with the logical identifier "23". A modifier is also defined to allow the client to be explicit about the intended type:

```
GET [base]/Observation?subject:Patient=23
```

This has the same effect as the previous search. The modifier becomes useful when used with chaining as explained in the next section. Note: The `[type]` modifier can't be used with a reference to a resource found on another server, since the server would not usually know what type that resource has. However, since these are absolute references, there can be no ambiguity about the type.

In some cases, search parameters are defined with an implicitly limited scope. For example, [Observation \(observation.html\)](#) has an element `subject`, which is a reference to one of a number of types. This has a matching search parameter `subject`, which refers to any of the possible types. In addition to this, there is another search parameter `patient`, which also refers to `Observation.subject`, but is limited to only include references of type [Patient \(patient.html\)](#). When using the patient search parameter, there is no need to specify `":Patient"` as a modifier, or `"Patient/"` in the search value, as this must always be true.

References are also allowed to have an `identifier`. The modifier `:identifier` allows for searching by the identifier rather than the literal reference:

```
GET [base]/Observation?subject:identifier=http://acme.org/fhir/identifier/mrn|123456
```

This is a search for all observations that reference a patient by a particular patient MRN. When the `:identifier` modifier is used, the search value works as a [token search](#). The `:identifier` modifier is not supported on canonical elements since they do not have an identifier separate from the reference itself.

Chaining is not supported when using the `:identifier` modifier, nor are chaining, includes or reverse includes supported for reference elements that do not have a `reference` element.

The reference search parameter is mostly used for resource elements of type `Reference` or `canonical`. However, it is also be used to search resource elements of type `Resource` - i.e. where one resource is directly nested within another - see the [Bundle search parameters \(bundle.html#search\)](#) 'message' and 'composition' as an example of this.

3.1.1.4.13 References and Versions

Elements of type [Reference \(references.html#Reference\)](#) may contain a [versioned \(references.html#versions\)](#) reference:

```
<evidence>
  <reference value="Observation/123/_history/234234"/>
</evidence>
```

When searching on versioned references, the following rules apply:

- If a resource has a reference that is versioned, and chaining is performed, the criteria should ideally be evaluated against the version referenced, but most systems will not be capable of this because search is only defined to function against the current version of a resource
- Where a search does not act on the referenced version, search results SHOULD contain a `OperationOutcome` with a warning that indicates the discrepancy
- If a resource has a reference that is versioned and `_include` is performed, the specified version SHOULD be provided.

Elements of type [canonical \(references.html#canonical\)](#) may contain a [version specific \(references.html#versions\)](#) reference, but this version is different in both meaning and format to version specific references that might be found in a [Reference \(references.html#Reference\)](#):

```
<valueSet value="http://hl7.org/fhir/ValueSet/example|3.0"/>
```

This version is a reference to the business version of the resource.

For canonical references, servers SHOULD support searching by Canonical URLs, and SHOULD support automatically detecting a `[[version]]` portion as part of the search parameter and interpreting that portion as a search on the business version of the target resource. The modifier `:below` is used with canonical references, to control whether the version is considered in the search. The search:

```
GET {base}/Observation?definition:below=http:http://acme.com/some-profile
```

matches all of these element values:

- `http://acme.com/some-profile|1.0`
- `http://acme.com/some-profile|1.1`
- `http://acme.com/some-profile|2.0`

The search:

```
GET {base}/Observation?definition:below=http:http://acme.com/some-profile|1
```

matches the first two element values.

3.1.1.4.14 Searching Hierarchies

Some references are [circular \(references.html#circular\)](#) - that is, the reference points to another resource of the same type. When the reference establishes a strict hierarchy, the modifiers `:above` and `:below` may be used to search transitively through the hierarchy:

```
GET [base]/Procedure?location:below=42
```

This search returns not only all procedures that occurred at location with id 42, but also any procedures that occurred in locations that are part of location with id 42.

```
GET [base]/MedicationAdministration?encounter:above=21
```

Returns all medication administrations that happened during encounter with id 21 or during any "parent" encounter of that encounter.

Servers indicate that :above/:below is supported on a search parameter by defining them as [Modifiers \(searchparameter-definitions.html#SearchParameter.modifier\)](#) on the Search Parameter definition.

3.1.1.4.15 Chained parameters

In order to save a client from performing a series of search operations, reference parameters may be "chained" by appending them with a period (.) followed by the name of a search parameter defined for the target resource. This can be done recursively, following a logical path through a graph of related resources, separated by . . For instance, given that the resource [DiagnosticReport \(diagnosticreport.html\)](#) has a search parameter named *subject*, which is usually a reference to a [Patient \(patient.html\)](#) resource, and the Patient resource includes a parameter *name* which searches on patient name, then the search

```
GET [base]/DiagnosticReport?subject.name=peter
```

is a request to return all the lab reports that have a subject whose name includes "peter". Because the Diagnostic Report subject can be one of a set of different resources, it's necessary to limit the search to a particular type:

```
GET [base]/DiagnosticReport?subject:Patient.name=peter
```

This request returns all the lab reports that have a subject which is a patient, whose name includes "peter".

Note that chained parameters are applied independently to the target resource. For example,

```
GET Patient?general-practitioner.name=Joe&general-practitioner.address-state=MN
```

may return Patients cared for by Joe from CA and Jane from MN: no one practitioner need satisfy both conditions. E.g. the chains are evaluated separately. For use cases where the joins must be evaluated in groups, there are either [Composite](#) search parameters, or the [filter](#) parameter.

Advanced Search Note: Where a chained parameter searches a resource reference that may have more than one type of resource as its target, the parameter chain may end up referring to search parameters with the same name on more than one kind of resource at once. Servers SHOULD reject a search where the logical id refers to more than one matching resource across different types. For example, the client has to specify the type explicitly using the syntax in the second example above.

3.1.1.4.16 Reverse Chaining

The `_has` parameter provides limited support for reverse chaining - that is, selecting resources based on the properties of resources that refer to them (instead of chaining, above, where resources can be selected based on the properties of resources that they refer to). Here is an example of the `_has` parameter:

```
GET [base]/Patient?_has:Observation:patient:code=1234-5
```

This requests the server to return Patient resources, where the patient resource is referred to by at least one Observation where the observation has a code of 1234, and where the Observation refers to the patient resource in the patient search parameter.

"Or" searches are allowed (e.g. GET [base]/Patient?_has:Observation:patient:code=123,456), and multiple `_has` parameters are allowed (e.g. GET [base]/Patient?_has:Observation:patient:code=123&_has:Observation:patient:code=456). Note that each `_has` parameter is processed independently of other `_has` parameters.

The `_has` parameter can be chained, like this:

```
GET [base]/Patient?_has:Observation:patient:_has:AuditEvent:entity:user=MyUserId
```

Fetch all the patients that have an Observation where the observation has an audit event from a specific user.

3.1.1.4.17 Composite Search Parameters

Composite search parameters support joining single values with a `$`. For example, the result of the search operation is the intersection of the resources that match the criteria specified by each individual search parameter. If a parameter repeats, such as `/Patient?language=FR&language=NL`, then this matches a patient who speaks both languages. This is known as an AND search parameter, since the server is expected to respond only with results which match both values.

If, instead, the search is to find patients that speak either language, then this is a single parameter with multiple values, separated by a `,`. For example, `/Patient?language=FR,NL`. This is known as an OR search parameter, since the server is expected to respond with results which match either value. Every search parameter may be used with comma-separated values in this fashion; this includes the use of search parameters with modifiers, such as `?code:text=this,that`.

AND parameters and OR parameters may also be combined, for example: `/Patient?language=FR,NL&language=EN` would refer to any patient who speaks English, as well as either French or Dutch.

This approach allows for simple combinations of and/or values, but doesn't allow a search based on a pair of values, such as all observations with a sodium value >150 mmol/L (particularly as the end criteria of a chained search), or searching on `Group.characteristic` where you need find a combination of key/value, not an intersection of separate matches on key and value. Another example is spatial coordinates when doing geographical searches.

To allow these searches, a resource may also specify *composite* parameters that take sequences of single values that match other defined parameters as an argument. The matching parameter of each component in such a sequence is documented in the definition of the parameter. These sequences are formed by joining the single values with a `$`. Note: This sequence is a single value and itself can be composed into a set of values, so that, for example, multiple matching characteristic-value parameters can be specified as `GET [base]/Group?characteristic-value=gender$mixed,owner$Eve`.

Note: Modifiers are not used on composite parameters.

Examples of using composite parameters:

Search	Description
<code>GET [base]/DiagnosticReport?result.code-value-quantity=http://loinc.org 2823-3\$gt5.4 http://unitsofmeasure.org mmol/L</code>	Search for all diagnostic reports that contain an observation with a potassium value of >5.4 mmol/L (UCUM)
<code>GET [base]/Observation?component-code-value-quantity=http://loinc.org 8480-6\$lt60</code>	Search for all the observations with a systolic blood pressure < 60 . Note that in this case, the unit is assumed (everyone uses mmHg)
<code>GET [base]/Group?characteristic-value=gender\$mixed</code>	Search for all groups that have a characteristic "gender" with a text value of "mixed"
<code>GET [base]/Questionnaire?context-type-value=focus\$http://snomed.info/sct 408934002</code>	Search for all questionnaires that have a clinical focus = "Substance abuse prevention assessment (procedure)"

3.1.1.4.18 Handling Missing Data

Consider the case of searching for all AllergyIntolerance resources:

```
GET [base]/AllergyIntolerance?clinical-status=http://terminology.hl7.org/CodeSystem/allergyintolerance-clinical|active
```

This search will only return resources that have a value for clinicalStatus:

```
{
  "resourceType" : "AllergyIntolerance",
  "clinicalStatus": {
    "coding": [
      {
        "system": "http://terminology.hl7.org/CodeSystem/allergyintolerance-clinical",
        "code": "active"
      }
    ]
  }
}
```

Resources missing a clinicalStatus will not be returned. This is probably unsafe - it would not usually be appropriate to ignore AllergyIntolerance warnings with an unknown clinical status, and only return resources with an explicit clinicalStatus. Instead, it might be desired to return AllergyIntolerance resources with either an explicit value for clinicalStatus, or none:

```
GET [base]/AllergyIntolerance?clinical-status=http://terminology.hl7.org/CodeSystem/allergyintolerance-clinical|active
GET [base]/AllergyIntolerance?clinical-status:missing=true
```

Note that this is 2 separate queries. They can be [combined in a batch \(http.html#transaction\)](http://hl7.org/fhir/transaction), but not in a single operation. This query will always return an empty list, as no resource can satisfy both criteria at once:

```
GET [base]/AllergyIntolerance?clinical-status=http://terminology.hl7.org/CodeSystem/allergyintolerance-clinical|active&clinical-status:missing=true
```

There is no way to use the :missing modifier and mix with a value using the comma syntax documented above for composite search parameters.

An alternative approach is to use the [filter](#) parameter, for servers that support this parameter.

3.1.1.4.19 Escaping Search Parameters

In the rules described above, special rules are defined for the characters \$, , and | . As a consequence, if these characters appear in an actual parameter value, they must be differentiated from their use as separator characters. When any of these characters appear in an actual parameter value, they must be prepended by the character \ , which also must be used to prepend itself. Therefore, param=xxx\$xxx indicates that it is a composite parameter, while param=xx\\$xx indicates that the parameter has the literal value xx\$xx . The parameter value xx\xx is illegal, and the parameter value param=xx\\xx indicates a literal value of xx\xx . This means that:

```
GET [base]/Observation?code=a,b
```

is a request for any Observation that has a code of either a or b , whereas:

```
GET [base]/Observation?code=a\,b
```

is a request for any Observation that has a code of `a, b`.

This escaping is at a different level to the % encoding that applies to all URL parameters. Standard % escaping still applies, such that these URLs have the same meaning:

```
GET [base]/ValueSet?url=http://acme.org/fhir/ValueSet/123,http://acme.org/fhir/ValueSet/124,125
GET [base]/ValueSet?url=http://acme.org/fhir/ValueSet/123,http://acme.org/fhir/ValueSet/124%2CValueSet/125
```

As do these URLs:

```
GET [base]/ValueSet?url=http://acme.org/fhir/ValueSet/123,http://acme.org/fhir/ValueSet/124\,125
GET [base]/ValueSet?url=http%58%47%47acme%46org%47fhir%47ValueSet%47123%44http%58%47%47acme%46org%47fhir%47ValueSet%47124%92%46125
```

This specification defines this additional form of escape because the escape syntax using \ applies to all parameter values after they have been 'unescaped' on the server while being read from the HTTP headers.

3.1.1.4.20 Text Search Parameters

The special text search parameters, `_text` and `_content`, search on the narrative of the resource, and the entire content of the resource respectively. Just like string parameters using the `:text (search.html#modifiers)` modifier, these parameters SHOULD support a sophisticated search functionality of the type offered by typical text indexing services. The value of the parameter is a text-based search, which may involve searching multiple words with thesaurus and proximity considerations, and logical operations such as AND, OR etc. For example:

```
GET [base]/Condition?_text=(bone OR liver) and metastases
```

This request returns all Condition resources with the word "metastases" and either "bone" or "liver" in the narrative. The server MAY choose to search for related words as well.

Implementers could consider using the rules specified by the [OData specification for the \\$search parameter](http://docs.oasis-open.org/odata/odata/v4.0/cs01/part1-protocol/odata-v4.0-cs01-part1-protocol.html#TheSearchSystem) ([http://docs.oasis-open.org/odata/odata/v4.0/cs01/part1-protocol/odata-v4.0-cs01-part1-protocol.html# The \\$search System](http://docs.oasis-open.org/odata/odata/v4.0/cs01/part1-protocol/odata-v4.0-cs01-part1-protocol.html#TheSearchSystem)). Typical implementations would use Lucene, Solr, an SQL-based full text search, or some similar indexing service.

3.1.1.4.21 Special Parameters

A few parameters have the type 'special'. That indicates that the way this parameter works is unique to the parameter and described with the parameter. The general modifiers and comparators do not apply, except as stated in the description.

Implementers will generally need to do special implementations for these parameters. These parameters are special:

- `filter (search filter.html)` (all resources)
- `near` on `Location (location.html#search)`.

3.1.1.4.22 Searching by list

The `_list` parameter allows for the retrieval of resources that are referenced by a `List (list.html)` resource.

```
GET [base]/Patient?_list=42
```

This request returns all Patient resources that are referenced from the list found at `[base]/List/42` in `List.entry.item`. While it is possible to retrieve the list, and then iterate the entries in the list fetching each patient, using a list as a search criterion allows for additional search criteria to be specified. For instance:

```
GET [base]/Patient?_list=42&gender=female
```

This request will return all female patients in the list. The server can return the list referred to in the search parameter as an included resource, but is not required to do so. In addition, a system can support searching by lists by their logical function. For example:

```
GET [base]/AllergyIntolerance?patient=42&_list=$current-allergies
```

This request will return all allergies in patient 42's "Current Allergy List". The server returns all relevant AllergyIntolerance resources, and can also choose to return the list. For further information, refer to the [definition of "\\$current-allergies" \(lifecycle.html#current\)](#), and the [List Operation "Find" \(list-operations.html#find\)](#). Note: Servers are not required to make these lists available to the clients as list resources, but may choose to do so.

3.1.1.4.23 Advanced filtering

The search mechanism described above is flexible, and easy to implement for simple cases, but is limited in its ability to express combination queries. To complement this mechanism, the `"_filter"` search expression parameter can be used.

For example, "Find all the observations for patient with a name including `peter` that have a LOINC code `1234-5`":

```
GET [base]/Observation?code=http://loinc.org|1234-5&subject.name=peter
```

Using the `_filter` parameter, the search would be expressed like this:

```
GET [base]/Observation?_filter=code eq http://loinc.org|1234-5 and subject.name co "peter"
```

The `_filter` parameter is described in detail on the ["_ Filter Parameter" page \(search_filter.html\)](#).

3.1.1.4.24 Specifying the type of Resource

Normally, a search is initiated against a known type of resource, e.g.

```
GET [base]/Observation?params...
```

However, in some circumstances, a search is executed where there is no fixed type of resource:

- Using search across all resource types (`GET [base]?params...`)
- Using search with [messaging \(messaging.html\)](#)
- Some search specifications inside other services e.g. decision support usage

In these circumstances, the search criteria may need to specify one or more resource types that the search applies to. This can be done by using the `_type` parameter:

```
GET [base]/?_type=Observation,Condition&other params...
```

If no type is specified, the only search parameters that be can be used in global search like this are the base parameters that apply to all resources. If multiple types are specified, any search parameters shared across the entire set of specified resources may be used (see [search parameter registry](#) ([searchparameter-registry.html#common](#))).

Technically, the `_type` parameter is a token parameter on the [Resource Types](#) ([valueset-resource-types.html](#)) Value Set.

3.1.1.5 Managing Returned Resources

3.1.1.5.1 Sorting

The client can indicate which order to return the results by using the parameter `_sort` , which can contain a comma-separated list of sort rules in priority order:

```
GET [base]/Observation?_sort=status,-date,category
```

Each item in the comma separated list is a search parameter, optionally with a '-' prefix. The prefix indicates decreasing order; in its absence, the parameter is applied in increasing order.

Notes:

- When sorting, the actual sort value used is not returned explicitly by the server for each resource, just the resource contents.
- To sort by relevance, use `_score` .
- The server returns the sort it performs as part of the returned search parameters (see [below](#)).
- A search parameter can refer to an element that repeats, and therefore there can be multiple values for a given search parameter for a single resource. In this case, the sort is based on the item in the set of multiple parameters that comes earliest in the specified sort order when ordering the returned resources.
- When sorting on string search parameters, sorting SHOULD be performed on a case-insensitive basis. Accents may either be ignored or sorted as per realm convention.
- This specification does not specify exacts rules for consistency of sorting across servers. In general, this is deemed to be not as essential as consistency of filtering (though even that is a little variable). The purpose of sorting is to provide data in a "reasonable" order for end-users. "Reasonable" may vary by use case and realm, particularly for accented characters.

The `_total` parameter has the status [trial-use \(versions.html#std-process\)](#) pending real world experience of it's use.

3.1.1.5.2 Total number of matching resources

The return [Bundle](#) ([bundle.html](#)) has an element `total` which is the number of resources that match the search parameters.

Note that `Bundle.total` represents the total number of matches, not how many resources are returned in a particular response (see [paging](#), immediately below).

Providing a precise number of matching resources may be onerous for the server, depending on how the server is designed. To help reduce the server load, a client can provide the parameter `_total` to indicate it's preference with regard to the total, which can have one of the following values:

none	There is no need to populate the total count; the client will not use it
estimate	A rough estimate of the number of matching resources is sufficient
accurate	The client requests that the server provide an exact total of the number of matching resources

The `Bundle.total` element is still optional, and the servers can ignore the `_total` parameter: it is just an optimization hint, that might possibly save the server some work.

3.1.1.5.3 Page Count

In order to keep the load on clients, servers and the network minimized, the server may choose to return the results in a series of pages. The search result set contains the URLs that the client uses to request additional pages from the search set. For a simple RESTful search, the page links are contained in the returned bundle as links (<http.html#paging>).

Typically, a server will provide its own parameters in the links that it uses to manage the state of the search as pages are retrieved. These parameters do not need to be understood or processed by the client.

The parameter `_count` is defined as an instruction to the server regarding how many resources should be returned in a single page. Servers SHALL NOT return more resources than requested, even if they don't support paging, but may return less than the client requested. The server should repeat the original `_count` parameter in its returned page links so that subsequent paging requests honor the original `_count`. Note: It is at the discretion of the search engine as to how to handle ongoing updates to the resources while the search is proceeding.

Note: The combination of `_sort` and `_count` can be used to return only the latest resource that meets a particular criteria - set the criteria, and then sort by date in descending order, with `_count=1`. This way, the last matching resource will be returned.

if `_count` has the value 0, this shall be treated the same as `_summary=count`: the server returns a bundle that reports the total number of resources that match in `Bundle.total`, but with no entries, and no prev/next/last links. Note that the `Bundle.total` only include the total number of matching resources. It does not count extra resources such as [OperationOutcome \(operationoutcome.html\)](operationoutcome.html) or [included](#) resources that may also be returned. In the same way, the `_count` parameter only applies to resources with `entry.search.mode = search`, and does not include included resources or operation outcomes.

The `_count` parameter has no impact on the value of `Bundle.total` as the latter represents the total number of matches, not how many are returned in a single Bundle response.

3.1.1.5.4 Including other resources in result (`_include` and `_revinclude`)

Clients may request that the engine return resources related to the search results, in order to reduce the overall network delay of repeated retrievals of related resources. This is useful when the client is searching on a clinical resource, but for every such resource returned, the client will also need the subject (patient) resource that the clinical resource refers to. The client can use the `_include` parameter to indicate that the subject resources be included in the results. An alternative scenario is where the client wishes to fetch a particular resource, and any resources that refer to it. For example, the client may wish to fetch a MedicationRequest, and any provenance resources that refer to the prescription. This is known as a reverse include, and is specified by providing a `_revinclude` parameter.

Both `_include` and `_revinclude` are based on search parameters, rather than paths in the resource, since joins, such as [chaining](#), are already done by search parameter.

Each `_include` parameter specifies a search parameter to join on:

```
GET [base]/MedicationRequest?_include=MedicationRequest:patient
GET [base]/MedicationRequest?_revinclude=Provenance:target
```

The first search requests all matching MedicationRequests, to include any patient that the medication prescriptions in the result set refer to. The second search requests all matching prescriptions, return all the provenance resources that refer to them.

Parameter values for both `_include` and `_revinclude` have three parts, separated by a `:` character:

1. The name of the source resource from which the join comes
2. The name of the search parameter which must be of type *reference*
3. (Optional) A specific type of target resource (for when the search parameter refers to multiple possible target types)

`_include` and `_revinclude` parameters do not include multiple values. Instead, the parameters are repeated for each different include criteria.

For each returned resource, the server identifies the resources that meet the criteria expressed in the join, and adds to the results, with the [entry.search.mode](#) ([bundle-definitions.html#Bundle.entry.search.mode](#)) set to "include" (in some searches, it is not obvious which resources are matches, and which are includes). If there is no reference, or no matching resource, the resource cannot be retrieved (e.g. on a different server), then the resource is omitted, and no error is returned.

The inclusion process can be iterative, if (and only if) the modifier `:iterate` is included. For example, this search returns all [Medication Request](#) ([medicationrequest.html](#)) resources and their [prescribing Practitioner](#) ([practitioner.html](#)) Resources for the matching [Medication Dispense](#) ([medicationdispense.html](#)) resources:

```
GET [base]/MedicationDispense?_include=MedicationDispense:prescription
    &_include:iterate=MedicationRequest:performer&criteria...
```

This technique applies to circular relationships as well. For example, the first of these two searches includes any related observations to the target relationships, but only those directly related. The second search asks for the `_include` based on `related` parameter to be executed iteratively, so it will retrieve observations that are directly related, and also any related observations to any other included observation.

```
GET [base]/Observation?_include=Observation:related-target&criteria...
GET [base]/Observation?_include:iterate=Observation:related-target&criteria...
```

Both `_include` and `_revinclude` use the wild card "*" for the search parameter name, indicating that any search parameter of type=reference be included. Though both clients and servers need to take care not to request or return too many resources when doing this. Most notably, using iterative wildcards inclusions might lead to the retrieval of the full patient's record, or even more than that: resources are organized into an interlinked network and broad `_include` paths may eventually traverse all possible paths on the server. For servers, these iterative and wildcard `_include` s are demanding and may slow the search response time significantly.

It is at the server's discretion how deep to iteratively evaluate the inclusions. Servers are expected to limit the number of iterations done to an appropriate level and are not obliged to honor requests to include additional resources in the search results. Because iterative search is generally resource intensive, it is not the default behavior.

When search results are paged, each page of search results should include the matching includes for the resources in each page, so that each page stands alone as a coherent package.

Note: when considering using `_include` and `_revinclude`, implementers should also consider whether using [GraphQL](#) ([graphql.html](#)) and/or [GraphDefinition](#) ([graphdefinition.html](#)) are more appropriate approaches in their context.

3.1.1.5.5 Contained Resources

By default, search results only include resources that are not contained in other resources. A chained condition will be evaluated inside contained resources. To illustrate this, consider a [MedicationRequest](#) resource that has a contained [Medication](#) resource specifying a custom formulation that has ingredient with a `itemCodeableConcept` "abc" in "http://acme.com./medications". In this case, a search:

```
GET MedicationRequest?medication.ingredient-code=abc
```

will include the [MedicationRequest](#) resource in the results. However, this search:

```
GET Medication?ingredient-code=abc
```

will not include the contained Medication resource in the results, since either the wrong type of resource would be returned, or the contained resource would be returned without its container resource, which provides context to the contained resource.

Clients can modify this behavior using the `_contained` parameter, which can have one of the following values:

- false (default): Do not return contained resources
- true: return only contained resources
- both: return both contained and non-contained (normal) resources

When contained resources are being returned, the server should return either the container resource, or the contained resource alone. The client can specify which by using the `_containedType` parameter, which can have one of the following values:

- container (default): Return the container resources
- contained: return only the contained resource

When returning a container resource, the server simply puts this in the search results:

```
<Bundle>
...
<entry>
  <resource>
    <MedicationRequest>
      <id value="23">
        ....
        <contained>
          <Medication>
            <id value="m1">
              ...
            </Medication>
          </contained>

        </MedicationRequest>
      </resource>
    <search>
      <mode value="match"/>
    </search>
  </entry>
</Bundle>
```

In the case of returning container resources, the server SHALL populate the `entry.search.mode` element, as shown, so that the client can pick apart matches and includes (since the usual approach of doing it by type might not work).

If the return type is the contained resource, this must be done slightly differently:

```
<Bundle>
  ...
  <entry>
    <fullUrl value="http://example.com/fhir/MedicationRequest/23#m1"/>
    <resource>
      <Medication>
        <id value="m1">
          ...
        </Medication>
      </resource>
    <search>
      <mode value="match"/>
    </search>
  </entry>
</Bundle>
```

In this case, the fullUrl informs the client that this is a contained resource, along with indicating the identity of the containing resource.

3.1.1.5.6 External References

If the `_include` path selects a reference that refers to a resource on another server, the server can elect to include that resource in the search results for the convenience of the client.

If the `_include` path selects a reference that refers to an entity that is not a Resource, such as an image attachment, the server may also elect to include this in the returned results as a [Binary \(binary.html\)](#) resource. For example, the include path may point to an attachment which is by reference, like this:

```
<content>
  <contentType>image/jpeg</contentType>
  <url>http://example.org/images/2343434/234234.jpg</url>
</content>
```

The server can retrieve the target of this reference, and add it to the results for the convenience of the client.

3.1.1.5.7 Paging

When returning paged results for a search with `_include` resources, all `_include` resources that are related to the primary resources returned for the page SHOULD also be returned as part of that same page, even if some of those resource instances have previously been returned on previous pages. This approach allows both sender and receiver to avoid caching results of other pages.

3.1.1.5.8 Summary

The client can request the server to return only a portion of the resources by using the parameter `_summary` :

```
GET [base]/ValueSet?_summary=true
```

The `_summary` parameter requests the server to return a subset of the resource. It can contain one of the following values:

true	Return a limited subset of elements from the resource. This subset SHOULD consist solely of all supported elements that are marked as "summary" in the base definition of the resource(s) (see ElementDefinition.isSummary (elementdefinition-definitions.html#ElementDefinition.isSummary)))
text	Return only the "text" element, the 'id' element, the 'meta' element, and only top-level mandatory elements

<u>data</u>	Remove the text element
<u>count</u>	Search only: just return a count of the matching resources, without returning the actual matches
<u>false</u>	Return all parts of the resource(s)

The intent of the `_summary` parameter is to reduce the total processing load on server, client, and resources between them such as the network. It is most useful for resources that are large, particularly ones that include images or elements that may repeat many times. The purpose of the summary form is to allow a client to quickly retrieve a large set of resources, and let a user pick the appropriate one. The summary for an element is defined to allow a user to quickly sort and filter the resources, and typically omit important content on the basis that the entire resource will be retrieved when the user selects a resource.

Servers are not obliged to return just a summary as requested. There are only a limited number of summary forms defined for resources in order to allow servers to store the summarized form(s) in advance. Servers **SHOULD** mark the resources with the tag **SUBSETTED** ([v3/SecurityIntegrityObservationValue/cs.html#SUBSETTED](#)) to ensure that the incomplete resource is not accidentally used to overwrite a complete resource.

Note that the `_include` and `_revinclude` parameters cannot be mixed with `_summary=text`.

Implementation Note: There is some question about the inclusion of extensions in the summary. Additional rules may be made around this in the future.

3.1.1.5.9 Elements

If one of the summary views defined above is not appropriate, a client can request a specific set of elements be returned as part of a resource in the search results using the `_elements` parameter:

```
GET [base]/Patient?_elements=identifier,active,link
```

The `_elements` parameter consists of a comma-separated list of base element names such as, elements defined at the root level in the resource. Only elements that are listed are to be returned. Clients **SHOULD** list all mandatory and modifier elements in a resource as part of the list of elements. The list of elements does not apply to [included resources](#).

Servers are not obliged to return just the requested elements. Servers **SHOULD** always return mandatory elements whether they are requested or not. Servers **SHOULD** mark the resources with the tag **SUBSETTED** ([v3/SecurityIntegrityObservationValue/cs.html#SUBSETTED](#)) to ensure that the incomplete resource is not actually used to overwrite a complete resource.

3.1.1.5.10 Relevance

Where a search specifies a non-deterministic sort, the search algorithm may generate a ranking score to indicate which resources meet the specified criteria better than others. The server can return this score in [entry.score \(bundle-definitions.html#Bundle.entry.score\)](#):

```
<entry>
  <score value=".45"/>
  <Patient>
    ... patient data ...
  </Patient>
</entry>
```

The score is a decimal number with a value between (and including) 0 and 1, where 1 is best match, and 0 is least match.

3.1.1.6 Server Conformance

In order to allow the client to be confident about what search parameters were used as criteria by the server, the server SHALL return the parameters that were actually used to process the search. Applications processing search results SHALL check these returned values where necessary. For example, if the server did not support some of the filters specified in the search, a client might manually apply those filters to the retrieved result set, display a warning message to the user or take some other action.

In the case of a RESTful search, these parameters are encoded in the self link in the bundle that is returned:

```
<link>
  <relation value="self"/>
  <url value="http://example.org/Patient?name=peter"/>
</link>
```

In other respects, servers have considerable discretion with regards to supporting search:

- Servers can choose which parameters to support (other than `_id` above).
- Servers can choose when and where to implement parameter chaining, and when and where they support the `_include` parameter.
- Servers can declare additional parameters in the profiles referenced from their Capability statements. Servers should define search parameters starting with a "-" character to ensure that the names they choose do not clash with future parameters defined by this specification.
- Servers are not required to enforce case sensitivity on parameter names, though the names are case sensitive (and URLs are generally case-sensitive).
- Servers may choose how many results to return, though the client can use `_count` as above
- Servers can choose how to sort the return results, though they SHOULD honor the `_sort` parameter.

3.1.1.7 Advanced Search

The search framework described above is a useful framework for providing a simple search based on indexed criteria, but more sophisticated query capability is needed to handle precise queries, complex decision support-based requests, and direct queries that have human resolution.

More advanced search operations are specified by the `_query` parameter:

```
GET [base]/Patient?_query=name&parameters...
```

The `_query` parameter names a custom search profile that describes a specific query operation. The named query may define additional named parameters that are used with that particular named query. Servers can define their own additional named queries to meet their own uses using an [OperationDefinition \(operationdefinition.html\)](#).

There can only ever be one `_query` parameter in a set of search parameters. Servers processing search requests SHALL refuse to process a search request if they do not recognize the `_query` parameter value.

3.1.1.8 Search Result Currency

The results of a search operation are only guaranteed to be current at the instant the operation is executed. After the operation is executed, ongoing actions performed on the resources against which the search was executed will render the results increasingly stale. The significance of this depends on the nature of the search, and the kind of use that is being made of the results.

This is particularly relevant when the server is returning the results in a series of pages. It is at the discretion of the search engine of how to handle ongoing updates to the resources while the search is proceeding.

Performing a search operation does not change the set of resources on the server, with the exception of the creation of [Audit Event \(auditevent.html\)](#) resources auditing the search itself.

3.1.1.9 Summary Tables

Common Parameters defined for all resources:

Name	Type	Description	Paths
<u>id</u>	<u>token</u>	Resource id (not a full URL)	Resource.id
<u>lastUpdated</u>	<u>date</u>	Date last updated. Server has discretion on the boundary precision	Resource.meta.lastUpdated
<u>tag</u>	<u>token</u>	Search by a resource tag	Resource.meta.tag
<u>profile</u>	<u>uri</u>	Search for all resources tagged with a profile	Resource.meta.profile
<u>security</u>	<u>token</u>	Search by a security label	Resource.meta.security
<u>text</u>	<u>string</u>	Text search against the narrative	
<u>content</u>	<u>string</u>	Text search against the entire resource	
<u>list</u>	<u>string</u>	All resources in nominated list (by id, not a full URL)	
<u>query</u>	<u>string</u>	Custom named query	

Search Control Parameters:

Name	Type	Description	Allowable Content
<u>sort</u>	<u>string</u>	Order to sort results in (can repeat for inner sort orders)	Name of a valid search parameter
<u>count</u>	<u>number</u>	Number of results per page	Whole number
<u>include</u>	<u>string</u>	Other resources to include in the search results that search matches point to	SourceType:searchParam(:targetType)
<u>reinclude</u>	<u>string</u>	Other resources to include in the search results when they refer to search matches	SourceType:searchParam(:targetType)
<u>summary</u>	<u>string</u>	Just return the summary elements (for resources where this is defined)	true false (false is default)
<u>contained</u>	<u>string</u>	Whether to return resources contained in other resources in the search matches	true false both (false is default)
<u>containedType</u>	<u>string</u>	If returning contained resources, whether to return the contained or container resources	container contained

Cross-map between search parameter types and Data types:

Data Type	<u>number</u>	<u>date</u>	<u>reference</u>	<u>quantity</u>	<u>uri</u>	<u>string</u>	<u>token</u>
Primitive Types							
<u>base64Binary</u> (datatypes.html#base64Binary).	Not used in search						
<u>boolean</u> (datatypes.html#boolean).	N	N	N	N	N	N	Y . true/false (System = http://hl7.org/fhir/special-values but not usually needed)
<u>canonical</u> (datatypes.html#canonical).	N	N	Y	N	Y	N	N
<u>code</u> (datatypes.html#code).	N	N	N	N	N	N	Y . (System, if desired, is defined in the underlying value set for each code)
<u>date</u> (datatypes.html#date).	N	Y	N	N	N	N	N
<u>dateTime</u> (datatypes.html#dateTime).	N	Y	N	N	N	N	N

<u>decimal</u> (datatypes.html#decimal).	Y	N	N	N	N	N	N
<u>id</u> (datatypes.html#id).	N	N	N	N	N	N	Y
<u>instant</u> (datatypes.html#instant).	N	Y	N	N	N	N	N
<u>integer</u> (datatypes.html#integer).	Y	N	N	N	N	N	N
<u>markdown</u> (datatypes.html#markdown).	Not used in search						
<u>oid</u> (datatypes.html#oid).	Not used in search (but see uri)						
<u>positiveInt</u> (datatypes.html#positiveInt).	Not used in search (but see integer)						
<u>string</u> (datatypes.html#string).	N	N	N	N	N	Y	Y
<u>time</u> (datatypes.html#time).	Not used in search						
<u>unsignedInt</u> (datatypes.html#unsignedInt).	Not used in search (but see integer)						
<u>uri</u> (datatypes.html#uri).	N	N	Y	N	Y	N	N
<u>url</u> (datatypes.html#url).	Not used in search (but see uri)						
<u>uuid</u> (datatypes.html#uuid).	Not used in search (but see uri)						
Data Types							
<u>Address</u> (datatypes.html#Address).	N	N	N	N	N	Y search on any string element in the address	N
<u>Age</u> (datatypes.html#Age).	N	N	N	Y	N	N	N
<u>Annotation</u> (datatypes.html#Annotation).	Not used in search						
<u>Attachment</u> (datatypes.html#Attachment).	Not used in search						
<u>CodeableConcept</u> (datatypes.html#CodeableConcept).	N	N	N	N	N	N	Y
<u>Coding</u> (datatypes.html#Coding).	N	N	N	N	N	N	Y
<u>Count</u> (datatypes.html#Count).	Not used in search						
<u>ContactPoint</u> (datatypes.html#ContactPoint).	N	N	N	N	N	N	Y
<u>Distance</u> (datatypes.html#Distance).	Not used in search						
<u>Duration</u> (datatypes.html#Duration).	N	N	N	Y	N	N	N
<u>HumanName</u> (datatypes.html#HumanName).	N	N	N	N	N	Y Search on any string element in the name	N
<u>Identifier</u> (datatypes.html#Identifier).	N	N	N	N	N	N	Y
<u>Money</u> (datatypes.html#Money).	N	N	N	Y	N	N	N
<u>Period</u> (datatypes.html#Period).	N	Y	N	N	N	N	N
<u>Quantity</u> (datatypes.html#Quantity).	N	N	N	Y	N	N	N
<u>Range</u> (datatypes.html#Range).	N	N	N	Y	N	N	N
<u>Ratio</u> (datatypes.html#Ratio).	Not used in search						
<u>Reference</u> (datatypes.html#Reference).	N	N	Y	N	N	N	N
<u>SampledData</u> (datatypes.html#base64Binary).	Not used in search						

Signature (datatypes.html#Signature)	Not used in search						
Timing (datatypes.html#Timing)	N	Y	N	N	N	N	N

© HL7.org 2011+. FHIR Release 4 (Technical Correction #1) (v4.0.1) generated on Fri, Nov 1, 2019 09:37+1100. [QA Page \(qa.html\)](#)
Links: [Search](#) [⚡ \(http://hl7.org/fhir/search.cfm\)](http://hl7.org/fhir/search.cfm) | [Version History \(history.html\)](#) | [Table of Contents \(toc.html\)](#) | [Credits \(credits.html\)](#) | [Compare to R3](#) [⚡](https://www.fhir.org/perl/htmldiff.pl?oldfile=http%3A%2F%2Fhl7.org%2Ffhir%2FSTU3%2Fsearch.html&newfile=http%3A%2F%2Fhl7.org%2Ffhir%2FR4%2Fsearch.html) [⚡](https://www.fhir.org/perl/htmldiff.pl?oldfile=http%3A%2F%2Fhl7.org%2Ffhir%2FSTU3%2Fsearch.html&newfile=http%3A%2F%2Fhl7.org%2Ffhir%2FR4%2Fsearch.html) |  [\(license.html\)](#) | [Propose a change](#) [⚡ \(http://hl7.org/fhir-issues\)](http://hl7.org/fhir-issues)